



# CP Template

Competitive Programming Reference · ECPC

C++ · 44 routines · 7 sections

Team reference — keep it, print it, win with it ❤️

# Contents

44 routines across 7 sections

## Setup

- Main Template — Single Test
- Main Template — Multi Test

## Graph

- Dijkstra
- Bellman-Ford
- Floyd-Warshall
- DSU (Union-Find)
- DFS
- DFS on Tree
- Cycle Detection — Directed
- Cycle Detection — Undirected
- Topological Sort
- Bipartite Check

## Number Theory

- Smallest Prime Factor (SPF)
- GCD Pair Counting (DP)

## Math

- Modular Arithmetic
- Combinatorics
- GCD / LCM / Ext-GCD
- Sequences & Sums
- Primes (Sieve)
- Prime Factorization
- Divisors
- Euler Totient
- Matrix Fast Power
- Fibonacci (Fast Doubling)
- Catalan Numbers
- Base Conversion
- Ceil/Floor & Digits

## Geometry

- Point & Vector
- Distances
- Orientation (CCW)
- Segment & Line Intersection
- Polygon Area
- Point in Polygon
- Convex Hull
- Angles & Rotation
- Shape Formulas

## Data Structures

- Ordered Set
- Ordered Multiset

## Bit Tricks

- Bitwise Operations
- Bitmask Techniques
- XOR Tricks
- Bit Counting & Log2
- Bit Hacks
- Binary → Gray Code

## Main Template — Single Test

Base solution skeleton

```
1 #include<bits/stdc++.h>
2 #define int long long
3 using namespace std;
4 void yalla_binaaaaaa(int t) {
5 }
6 signed main() {
7     ios::sync_with_stdio(0);
8     cin.tie(0), cout.tie(0);
9     int tc = 1;
10    // cin >> tc;
11    for(int t = 1; t <= tc; t++) {
12        yalla_binaaaaaa(t);
13    }
14    return 0;
15 }
```

## Main Template — Multi Test

Reads number of test cases

```
1 #include<bits/stdc++.h>
2 #define int long long
3 using namespace std;
4 void yalla_binaaaaaa(int t) {
5 }
6 signed main() {
7     ios::sync_with_stdio(0); cin.tie(0), cout.tie(0);
8     int tc = 1;
9     cin >> tc;
10    for(int t = 1; t <= tc; t++) {
11        yalla_binaaaaaa(t);
12    }
13
14    return 0;
15 }
```

**Dijkstra** Single-source shortest path (non-negative)

```
1 void dijkstra(int src) {
2     const int oo = 1e18;
3     dist.assign(n + 1, oo);
4     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
5
6     dist[src] = 0;
7     pq.emplace(dist[src], src);
8
9     while(not pq.empty()) {
10         auto [c, u] = pq.top();
11         pq.pop();
12
13         if(dist[u] < c) {
14             continue;
15         }
16
17         for(auto [v, w] : adj[u]) {
18             if(dist[u] + w < dist[v]) {
19                 dist[v] = dist[u] + w;
20                 pq.emplace(dist[v], v);
21             }
22         }
23     }
24 }
```

## Bellman-Ford Shortest path + negative cycle check

```
1 void BellmanFord(int src) { // O(N * M)
2     int oo = 1e18;
3     vector<int> dist(n, oo);
4     dist[src] = 0;
5     for(int i = 1; i < n; i++) {
6         for(auto [u, v, w] : edges) {
7             if(dist[u] != oo && dist[u] + w < dist[v]) {
8                 dist[v] = dist[u] + w;
9             }
10        }
11    }
12    bool inf = false; // check neg cycle
13    for(auto [u, v, w] : edges) {
14        if(dist[u] != oo && dist[u] + w < dist[v]) {
15            inf = true;
16            break;
17        }
18    }
19 }
```

## Floyd-Warshall All-pairs shortest path + neg cycle

```
1 void Floyd-Warshall() { // besd-0
2     const int oo = 1e18;
3     vector<vector<int>> dist(n, vector<int>(n, oo));
4     for(int u = 0; u < n; u++) {
5         dist[u][u] = 0;
6     }
7     for(int i = 0; i < m; i++) { // take input directed graph
8         int u, v, w;
9         cin >> u >> v >> w;
10        dist[u][v] = min(dist[u][v], w);
11    }
12    for(int k = 0; k < n; k++) { // all pairs shorts path
13        for(int u = 0; u < n; u++) {
14            for(int v = 0; v < n; v++) {
15                if(dist[u][k] != oo && dist[k][v] != oo) {
16                    dist[u][v] = min(dist[u][v], dist[u][k] + dist[k][v]);
17                }
18            }
19        }
20    }
21    for(int k = 0; k < n; k++) { // check negative cycle
22        if(dist[k][k] < 0) {
23            for(int u = 0; u < n; u++) {
24                for(int v = 0; v < n; v++) {
25                    if(dist[u][k] != oo && dist[k][v] != oo) {
26                        dist[u][v] = -oo;
27                    }
28                }
29            }
30        }
31    }
32    while(q--) {
33        int u, v;
34        cin >> u >> v;
35        if(dist[u][v] == oo) {
36            cout << "Impossible" << '\n';
37        }
38        else if(dist[u][v] == -oo) {
39            cout << "-Infinity" << '\n';
40        }
41        else {
42            cout << dist[u][v] << '\n';
43        }
44    }
45 }
```

## DSU (Union-Find) Disjoint set, union by size + path compression

```
1 struct DSU {
2     vector<int> leader, sz;
3     DSU(int n) {
4         leader.assign(n + 1, 0);
5         sz.assign(n + 1, 1);
6         iota(leader.begin(), leader.end(), 0);
7     }
8     int find(int u) {
9         if(leader[u] == u) return u;
10        return leader[u] = find(leader[u]);
11    }
12    bool merge(int u, int v) {
13        u = find(u), v = find(v);
14        if(u == v) return false;
15        if(sz[u] < sz[v]) {
16            swap(u, v);
17        }
18        leader[v] = u, sz[u] += sz[v];
19        return true;
20    }
21    int get_size(int u) {
22        return sz[find(u)];
23    }
24    bool connected(int u, int v) {
25        return find(u) == find(v);
26    }
27 };
```

## DFS Basic depth-first search

```
1 void dfs(int u) {
2     vis[u] = true;
3     for (int v : adj[u]) {
4         if (!vis[v]) {
5             dfs(v);
6         }
7     }
8 }
```

## DFS on Tree

DFS with parent tracking

```
1 void dfsTree(int u, int p) {
2     for (int v : adj[u]) {
3         if (v != p) {
4             dfsTree(v, u);
5         }
6     }
7 }
```

## Cycle Detection — Directed

Colors: 0 unvisited, 1 in-stack, 2 done

```
1 bool hasCycleDirected(int u) {
2     vis[u] = 1;
3     for (int v : adj[u]) {
4         if (vis[v] == 0) {
5             if (hasCycleDirected(v)) return true;
6         }
7         else if (vis[v] == 1) {
8             return true;
9         }
10    }
11    vis[u] = 2;
12    return false;
13 }
```

## Cycle Detection — Undirected

Parent-based back-edge check

```
1 bool CycleUndirected(int u, int parent) {
2     vis[u] = true;
3     for (int v : adj[u]) {
4         if (!vis[v]) {
5             if (hasCycleUndirected(v, u)) return true;
6         }
7         else if (v != parent) {
8             return true;
9         }
10    }
11    return false;
12 }
```

## Topological Sort DFS post-order (reverse at end)

```
1 vector<int> topo_order;
2 bool vis[N];
3 void topoSort(int u) {
4     vis[u] = true;
5     for (int v : adj[u]) {
6         if (!vis[v]) {
7             topoSort(v);
8         }
9     }
10    topo_order.push_back(u);
11 } // reverse(topo_order.begin(), topo_order.end());
```

## Bipartite Check 2-coloring via DFS

```
1 int color[N];
2 bool isBipartite(int u, int c) {
3     color[u] = c;
4     for (int v : adj[u]) {
5         if (color[v] == -1) {
6             if (!isBipartite(v, 1 - c)) return false;
7         }
8         else if (color[v] == color[u]) {
9             return false;
10        }
11    }
12    return true;
13 }
```

## Smallest Prime Factor (SPF) Sieve + $O(\log n)$ factorization

```

1  const int N = 1e7 + 5;
2  vector<int> spf(N);
3  // O(N Log Log N)
4  void build_spf() {
5      for (int i = 0; i < N; i++)
6          spf[i] = i;
7      for (int i = 2; i * i < N; i++) {
8          if (spf[i] == i) {
9              for (int j = i * i; j < N; j += i) {
10                 if (spf[j] == j)
11                     spf[j] = i;
12             }
13         }
14     }
15 } // O(Log n)
16 vector<int> factorize(int n) {
17     vector<int> factors;
18     while (n > 1) {
19         factors.push_back(spf[n]);
20         n /= spf[n];
21     }
22     return factors;
23 }

```

## GCD Pair Counting (DP) Count pairs with gcd = d

```

1  vector<int> dp(N); // dp[d] -> count pairs gcd(ai, aj) = d
2  for(int i = N-1; i >= 1; i--) {
3      int count_multiples = 0;
4      for(int j = i; j < N; j+=i)
5          count_multiples += freq[j];
6      dp[i] = (count_multiples * (count_multiples-1))>>1;
7      for(int j = i+i; j < N; j+=i)
8          dp[i] -= dp[j];
9  }

```

## Modular Arithmetic add / sub / mul / fast-power / inverse

```
1  const int MOD = 1e9 + 7;
2
3  inline int mod(int x) {
4      return ((x % MOD) + MOD) % MOD;
5  }
6
7  inline int add(int a, int b) {
8      return mod(mod(a) + mod(b));
9  }
10
11 inline int sub(int a, int b) {
12     return mod(mod(a) - mod(b));
13 }
14
15 inline int mul(int a, int b) {
16     return (mod(a) * mod(b)) % MOD;
17 }
18
19 int fp(int a, int b) {
20     a = mod(a);
21
22     int res = 1;
23
24     while (b > 0) {
25
26         if (b & 1)
27             res = mul(res, a);
28
29         a = mul(a, a);
30
31         b >>= 1;
32     }
33
34     return res;
35 }
36
37 inline int inv(int a) {
38     return fp(a, MOD - 2);
39 }
40
41 inline int divide(int a, int b) {
42     return mul(a, inv(b));
43 }
```

## Combinatorics nCr / nPr with factorial precompute

```
1  const int MOD = 1e9 + 7;
2  const int N = 1e6 + 5;
3  long long fact[N], invf[N];
4
5  // Precompute factorials and their modular inverses. Call once. Needs modpow.
6  void buildFact() {
7      fact[0] = 1;
8      for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i % MOD;
9      invf[N - 1] = modpow(fact[N - 1], MOD - 2); // Fermat inverse
10     for (int i = N - 1; i > 0; i--) invf[i - 1] = invf[i] * i % MOD;
11 }
12
13 // Choose r from n (order does NOT matter).
14 long long nCr(int n, int r) {
15     if (r < 0 || r > n) return 0;
16     return fact[n] * invf[r] % MOD * invf[n - r] % MOD;
17 }
18
19 // Arrange r from n (order MATTERS).
20 long long nPr(int n, int r) {
21     if (r < 0 || r > n) return 0;
22     return fact[n] * invf[n - r] % MOD;
23 }
24
25 /* ---- Examples ----
26     Want to pick a committee of 2 from 5 people? -> use nCr
27         buildFact(); nCr(5, 2) == 10
28     Want to hand gold & silver to 2 of 5?          -> use nPr
29         nPr(5, 2) == 20
30     Note: call buildFact() ONCE at program start.
31 */
```

## GCD / LCM / Ext-GCD Euclid + extended Euclid

```
1 // Greatest common divisor (Euclid). Built-in __gcd(a, b) also exists.
2 long long gcd(long long a, long long b) {
3     return b ? gcd(b, a % b) : a;
4 }
5
6 // Least common multiple. Divide first so we don't overflow.
7 long long lcm(long long a, long long b) {
8     return a / gcd(a, b) * b;
9 }
10
11 // Extended Euclid: finds x, y such that a*x + b*y = gcd(a, b).
12 // Used for modular inverse when MOD is NOT prime.
13 long long extgcd(long long a, long long b, long long &x, long long &y) {
14     if (!b) { x = 1; y = 0; return a; }
15     long long x1, y1;
16     long long g = extgcd(b, a % b, x1, y1);
17     x = y1;
18     y = x1 - (a / b) * y1;
19     return g;
20 }
21
22 /* ---- Examples ----
23     Want to simplify a fraction / common factor?    -> use gcd
24         gcd(12, 18) == 6
25     Want the smallest number divisible by both?      -> use lcm
26         lcm(4, 6) == 12
27     Want a modular inverse when MOD is NOT prime?   -> use extgcd
28         extgcd(3, 5, x, y): g = 1, with 3*2 + 5*(-1) = 1
29 */
```

## Sequences & Sums Arithmetic, geometric, sum formulas

```
1 // Sum  $1 + 2 + \dots + n = n(n+1)/2$ 
2 long long sumN(long long n) { return n * (n + 1) / 2; }
3
4 // Sum of squares  $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$ 
5 long long sumSq(long long n) { return n * (n + 1) * (2 * n + 1) / 6; }
6
7 // Sum of cubes  $1^3 + \dots + n^3 = (n(n+1)/2)^2$ 
8 long long sumCube(long long n) { long long s = n * (n + 1) / 2; return s * s; }
9
10 // Arithmetic series: first term a, common difference d, k terms.
11 long long arithSum(long long a, long long d, long long k) {
12     return k * (2 * a + (k - 1) * d) / 2;
13 }
14
15 // Geometric series  $a + a*r + \dots + a*r^{(k-1)}$  (integer,  $r \neq 1$ ).
16 long long geoSum(long long a, long long r, long long k) {
17     long long p = 1;
18     for (int i = 0; i < k; i++) p *= r; //  $r^k$ 
19     return a * (p - 1) / (r - 1);
20 }
21
22 /* ---- Examples ----
23 Want  $1 + 2 + \dots + n$  instantly?      -> use sumN
24     sumN(100) == 5050
25 Want sum of squares  $1^2..n^2$ ?          -> use sumSq
26     sumSq(3) == 14           //  $1 + 4 + 9$ 
27 Want an arithmetic series sum?         -> use arithSum
28     arithSum(2, 3, 4) == 26    //  $2 + 5 + 8 + 11$ 
29 Want a geometric series sum?           -> use geoSum
30     geoSum(1, 2, 4) == 15     //  $1 + 2 + 4 + 8$ 
31 */
```

## Primes (Sieve) All primes up to N + primality test

```
1 // Sieve of Eratosthenes: find all primes up to LIM in  $O(n \log \log n)$ .
2 const int LIM = 1e6 + 5;
3 bool comp[LIM];           // comp[i] == true -> i is NOT prime
4 vector<int> primes;
5 void sieve() {
6     for (int i = 2; i < LIM; i++) {
7         if (!comp[i]) {
8             primes.push_back(i);
9             for (long long j = (long long)i * i; j < LIM; j += i)
10                 comp[j] = true;
11         }
12     }
13 }
14
15 // Primality test for ONE number,  $O(\sqrt{n})$ .
16 bool isPrime(long long n) {
17     if (n < 2) return false;
18     for (long long i = 2; i * i <= n; i++)
19         if (n % i == 0) return false;
20     return true;
21 }
22
23 /* ---- Examples ----
24     Want ALL primes up to a limit fast? -> call sieve() once, read primes[]
25         sieve(); primes = {2, 3, 5, 7, 11, 13, ...}
26     Want to test if ONE number is prime? -> use isPrime
27         isPrime(97) == true ; isPrime(100) == false
28 */
```

## Prime Factorization Break n into prime powers

```
1 // Prime factorization by trial division,  $O(\sqrt{n})$ .
2 // Returns a list of (prime, exponent) pairs.
3 vector<pair<long long, int>> factorize(long long n) {
4     vector<pair<long long, int>> f;
5     for (long long p = 2; p * p <= n; p++) {
6         if (n % p == 0) {
7             int cnt = 0;
8             while (n % p == 0) { n /= p; cnt++; }
9             f.push_back({p, cnt});
10        }
11    }
12    if (n > 1) f.push_back({n, 1}); // whatever is left is a prime
13    return f;
14 }
15
16 /* ---- Examples ----
17     Want to break a number into its primes? -> use factorize
18     factorize(60) == {(2,2), (3,1), (5,1)}    // 60 = 2^2 * 3 * 5
19     factorize(97) == {(97,1)}                // 97 is prime
20 */
```

## Divisors Count & sum of divisors

```
1 // Number of divisors: if  $n = p_1^{a_1} * p_2^{a_2} * \dots$  then count =  $(a_1+1)(a_2+1)\dots$ 
2 long long countDivisors(long long n) {
3     long long cnt = 1;
4     for (long long p = 2; p * p <= n; p++) {
5         int e = 0;
6         while (n % p == 0) { n /= p; e++; }
7         cnt *= (e + 1);
8     }
9     if (n > 1) cnt *= 2; // one leftover prime -> exponent 1
10    return cnt;
11 }
12
13 // Sum of divisors: product of  $(p^{(a+1)} - 1) / (p - 1)$  over prime powers.
14 long long sumDivisors(long long n) {
15     long long sum = 1;
16     for (long long p = 2; p * p <= n; p++) {
17         if (n % p == 0) {
18             long long term = 1, pk = 1;
19             while (n % p == 0) { n /= p; pk *= p; term += pk; }
20             sum *= term;
21         }
22     }
23     if (n > 1) sum *= (1 + n);
24     return sum;
25 }
26
27 /* ---- Examples ----
28    Want HOW MANY divisors a number has? -> use countDivisors
29        countDivisors(12) == 6 // 1, 2, 3, 4, 6, 12
30    Want the SUM of all divisors? -> use sumDivisors
31        sumDivisors(12) == 28 // 1+2+3+4+6+12
32 */
```

## Euler Totient Count coprimes phi(n)

```
1 // Euler's Totient phi(n): how many integers in [1..n] are coprime with n.
2 // Formula: phi(n) = n * product over distinct primes p|n of (1 - 1/p).
3 long long phi(long long n) {
4     long long result = n;
5     for (long long p = 2; p * p <= n; p++) {
6         if (n % p == 0) {
7             while (n % p == 0) n /= p;
8             result -= result / p;      // multiply by (1 - 1/p)
9         }
10    }
11    if (n > 1) result -= result / n;    // last prime factor
12    return result;
13 }
14
15 /* ---- Examples ----
16    Want how many numbers <= n are coprime with n? -> use phi
17    phi(9) == 6    // 1, 2, 4, 5, 7, 8
18    phi(10) == 4    // 1, 3, 7, 9
19    Used in: Euler's theorem, modular inverse a^(phi(m)-1) mod m.
20 */
```

## Matrix Fast Power Linear recurrences / Fibonacci

```
1 // Matrix fast power - solves linear recurrences (e.g. Fibonacci)  $O(k^3 \log n)$ .
2 const int MOD = 1e9 + 7;
3 typedef vector<vector<long long>> Mat;
4
5 Mat mul(Mat &A, Mat &B) {
6     int n = A.size(), m = B[0].size(), k = B.size();
7     Mat C(n, vector<long long>(m, 0));
8     for (int i = 0; i < n; i++)
9         for (int j = 0; j < m; j++)
10             for (int t = 0; t < k; t++)
11                 C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
12     return C;
13 }
14
15 Mat matpow(Mat A, long long p) {
16     int n = A.size();
17     Mat R(n, vector<long long>(n, 0));
18     for (int i = 0; i < n; i++) R[i][i] = 1; // identity matrix
19     while (p > 0) {
20         if (p & 1) R = mul(R, A);
21         A = mul(A, A);
22         p >>= 1;
23     }
24     return R;
25 }
26
27 /* ---- Examples ----
28     Want  $F(n)$  for HUGE  $n$  (up to  $1e18$ )? -> use matpow
29     Mat base = {{1,1},{1,0}};
30     matpow(base, n)[0][1] == Fib(n)
31     Any recurrence  $f(n)=a*f(n-1)+b*f(n-2)$  fits by building its 2x2 matrix.
32 */
```

## Fibonacci (Fast Doubling) $F(n)$ in $O(\log n)$

```
1 // Fibonacci by fast doubling ->  $O(\log n)$ . Returns  $\{F(n), F(n+1)\}$ .
2 //  $F(2k) = F(k) * (2 * F(k+1) - F(k))$ 
3 //  $F(2k+1) = F(k)^2 + F(k+1)^2$ 
4 const int MOD = 1e9 + 7;
5 pair<long long, long long> fib(long long n) {
6     if (n == 0) return {0, 1};
7     auto p = fib(n >> 1);
8     long long a = p.first, b = p.second;
9     long long c = a * ((2 * b - a + MOD) % MOD) % MOD; //  $F(2k)$ 
10    long long d = (a * a + b * b) % MOD; //  $F(2k+1)$ 
11    if (n & 1) return {d, (c + d) % MOD};
12    return {c, d};
13 }
14
15 /* ---- Examples ----
16    Want  $F(n)$  for a HUGE  $n$  mod  $p$ ? -> use fib(n).first
17        fib(10).first == 55
18        sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
19 */
```

## Catalan Numbers Brackets, BSTs, triangulations

```
1 // Catalan number  $Cat(n) = C(2n, n) / (n + 1)$ .
2 // Needs nCr + modpow from the Combinatorics / Modular cards.
3 long long catalan(int n) {
4     return nCr(2 * n, n) * modpow(n + 1, MOD - 2) % MOD;
5 }
6
7 /* ---- Examples ----
8     $Cat(n)$  counts a LOT of things - use it when you need:
9        - # of valid bracket sequences with  $n$  pairs
10       - # of binary search trees with  $n$  nodes
11       - # of ways to triangulate a polygon with  $n+2$  sides
12       - # of monotonic lattice paths that stay below the diagonal
13    Example:
14        catalan(3) == 5
15        first Catalans: 1, 1, 2, 5, 14, 42, 132, ...
16 */
```

## Base Conversion

Decimal to/from any base + digits

```
1 // Convert a decimal number to a string in base b (2..36).
2 string toBase(long long n, int b) {
3     if (n == 0) return "0";
4     string s;
5     while (n > 0) {
6         int d = n % b;
7         s += (d < 10 ? '0' + d : 'A' + d - 10);
8         n /= b;
9     }
10    reverse(s.begin(), s.end());
11    return s;
12 }
13
14 // Parse a string written in base b back into a decimal number.
15 long long fromBase(string s, int b) {
16     long long n = 0;
17     for (char c : s) {
18         int d = (c <= '9' ? c - '0' : c - 'A' + 10);
19         n = n * b + d;
20     }
21     return n;
22 }
23
24 /* ---- Examples ----
25     Want a number in binary / hex / any base? -> use toBase
26         toBase(10, 2)    == "1010"
27         toBase(255, 16) == "FF"
28     Want to read a based string back to int?    -> use fromBase
29         fromBase("1010", 2) == 10
30 */
```

## Ceil/Floor & Digits Integer division, digit count/sum

```
1 // Integer ceil / floor division that also works for negative numbers.
2 long long ceilDiv(long long a, long long b) {
3     return a / b + ((a % b != 0) && ((a < 0) == (b < 0)));
4 }
5 long long floorDiv(long long a, long long b) {
6     return a / b - ((a % b != 0) && ((a < 0) != (b < 0)));
7 }
8
9 // Number of decimal digits of n.
10 int numDigits(long long n) { return n == 0 ? 1 : (int)log10(n) + 1; }
11
12 // Sum of the decimal digits of n.
13 int digitSum(long long n) {
14     int s = 0;
15     while (n > 0) { s += n % 10; n /= 10; }
16     return s;
17 }
18
19 /* ---- Examples ----
20     Want ceil(7/2) using integers (no double)? -> use ceilDiv
21         ceilDiv(7, 2) == 4 ; floorDiv(-7, 2) == -4
22     Want digit count / digit sum of a number? -> numDigits / digitSum
23         numDigits(12345) == 5 ; digitSum(12345) == 15
24 */
```

**Point & Vector** Core 2D point/vector operations

```
1 // 2D point / vector. Use double (or long double for extra precision).
2 struct P {
3     double x, y;
4     P operator + (P o) { return {x + o.x, y + o.y}; }
5     P operator - (P o) { return {x - o.x, y - o.y}; }
6     P operator * (double k) { return {x * k, y * k}; }
7     double dot(P o) { return x * o.x + y * o.y; } // dot a.b = |a||b|cos(t)
8     double cross(P o) { return x * o.y - y * o.x; } // cross = |a||b|sin(t)
(signed)
9     double norm() { return sqrt(x * x + y * y); } // Length |v|
10 };
11
12 /* ---- Examples ----
13     Want the LENGTH of vector (3,4)?          -> use norm()
14         P v{3,4}; v.norm() == 5
15     Want to know if b is LEFT/RIGHT of a?      -> use cross()
16         P a{1,0}, b{0,1}; a.cross(b) == 1 (b is CCW / to the Left)
17     Want to check two vectors are PERPENDICULAR? -> use dot() == 0
18         a.dot(b) == 0 -> yes, 90 degrees
19 */
```

## Distances Point-point, point-line, point-segment

```
1 // Euclidean distance between two points
2 double dist(P a, P b) { return (b - a).norm(); }
3
4 // Distance from point p to the INFINITE line through a, b.
5 double distToLine(P a, P b, P p) {
6     return fabs((b - a).cross(p - a)) / (b - a).norm();
7 }
8
9 // Distance from point p to the SEGMENT [a, b] (clamped to endpoints).
10 double distToSeg(P a, P b, P p) {
11     if ((b - a).dot(p - a) <= 0) return dist(a, p); // p is before a
12     if ((a - b).dot(p - b) <= 0) return dist(b, p); // p is after b
13     return distToLine(a, b, p); // foot lands on segment
14 }
15
16 /* ---- Examples ----
17     Want the straight gap between two points?      -> use dist
18         dist({0,0},{3,4}) == 5
19     Want the gap from a point to an INFINITE road? -> use distToLine
20         distToLine({0,0},{4,0},{2,3}) == 3
21     Want the gap from a point to a FINITE wall/edge? -> use distToSeg
22         distToSeg({0,0},{4,0},{6,0}) == 2 (clamped to endpoint b)
23 */
```

## Orientation (CCW) Cross-product left/right turn test

```
1 // Orientation of the ordered triplet (a, b, c) from the cross-product sign:
2 //  +1 -> counter-clockwise (left turn)
3 //  -1 -> clockwise          (right turn)
4 //   0 -> collinear
5 int orient(P a, P b, P c) {
6     double v = (b - a).cross(c - a);
7     if (v > 1e-9) return +1;
8     if (v < -1e-9) return -1;
9     return 0;
10 }
11
12 /* ---- Examples ----
13     Want to know which way you turn at a -> b -> c? -> use orient
14         orient({0,0},{1,0},{1,1}) == +1 (turning LEFT / CCW)
15         orient({0,0},{1,0},{1,-1}) == -1 (turning RIGHT / CW)
16     Want to test if 3 points lie on ONE line? -> orient == 0
17         orient({0,0},{1,0},{2,0}) == 0 (collinear)
18     Used inside: convex hull, segment intersection.
19 */
```

## Segment & Line Intersection

Do segments cross + line meet point

```
1 // Needs orient() and struct P. Is p (collinear with a,b) inside their box?
2 bool onSeg(P a, P b, P p) {
3     return min(a.x, b.x) - 1e-9 <= p.x && p.x <= max(a.x, b.x) + 1e-9 &&
4         min(a.y, b.y) - 1e-9 <= p.y && p.y <= max(a.y, b.y) + 1e-9;
5 }
6
7 // Do segments [a,b] and [c,d] intersect (touching counts)?
8 bool segInter(P a, P b, P c, P d) {
9     int o1 = orient(a, b, c), o2 = orient(a, b, d);
10    int o3 = orient(c, d, a), o4 = orient(c, d, b);
11    if (o1 != o2 && o3 != o4) return true;           // general proper crossing
12    if (o1 == 0 && onSeg(a, b, c)) return true;      // collinear touch cases
13    if (o2 == 0 && onSeg(a, b, d)) return true;
14    if (o3 == 0 && onSeg(c, d, a)) return true;
15    if (o4 == 0 && onSeg(c, d, b)) return true;
16    return false;
17 }
18
19 // Intersection POINT of two non-parallel lines a->b and c->d.
20 P lineInter(P a, P b, P c, P d) {
21     double t = (c - a).cross(d - c) / (b - a).cross(d - c);
22     return a + (b - a) * t;
23 }
24
25 /* ---- Examples ----
26    Want to know if two roads/segments CROSS?      -> use segInter
27        segInter({0,0},{4,4}, {0,4},{4,0}) == true (the diagonals cross)
28    Want the exact POINT where two lines meet?      -> use lineInter
29        lineInter({0,0},{4,4}, {0,4},{4,0}) == {2,2}
30 */
```

## Polygon Area Shoelace / Gauss signed area

```
1 // Signed area of a simple polygon (Shoelace / Gauss formula).
2 // |area| is the area; sign is + if vertices are CCW, - if CW.
3 double polygonArea(vector<P>& pts) {
4     double a = 0;
5     int n = pts.size();
6     for (int i = 0; i < n; i++) {
7         P p = pts[i], q = pts[(i + 1) % n]; // next vertex, wrapping around
8         a += p.cross(q);
9     }
10    return a / 2.0;
11 }
12
13 /* ---- Examples ----
14    Want the AREA of any polygon from its corners? -> use polygonArea (take fabs)
15        corners (0,0),(4,0),(4,4),(0,4) -> 16
16    Want to know if the corners are CCW or CW? -> Look at the SIGN
17        positive result => CCW order , negative => CW order
18 */
```

## Point in Polygon Ray-casting crossing-number test

```
1 // Point-in-polygon by ray casting (crossing number).
2 // Shoots a ray to the right; counts how many edges it crosses.
3 // Odd count -> inside, even -> outside.
4 bool inPolygon(vector<P>& poly, P p) {
5     int n = poly.size();
6     bool in = false;
7     for (int i = 0, j = n - 1; i < n; j = i++) {
8         P a = poly[i], b = poly[j];
9         // does edge (a,b) straddle the horizontal line y = p.y ?
10        if ((a.y > p.y) != (b.y > p.y) &&
11            p.x < (b.x - a.x) * (p.y - a.y) / (b.y - a.y) + a.x)
12            in = !in; // ray crosses this edge -> flip
13    }
14    return in;
15 }
16
17 /* ---- Examples ----
18    Want to know if a point is INSIDE a region/map? -> use inPolygon
19        region = (0,0),(4,0),(4,4),(0,4)
20        inPolygon(region, {2,2}) == true    (inside)
21        inPolygon(region, {5,2}) == false   (outside)
22 */
```

## Convex Hull Andrew's monotone chain $O(n \log n)$

```
1 // Convex hull via Andrew's monotone chain ->  $O(n \log n)$ .
2 // Returns hull vertices in counter-clockwise order (no repeated points).
3 vector<P> convexHull(vector<P> p) {
4     sort(p.begin(), p.end(), [](P a, P b) {
5         return a.x < b.x || (a.x == b.x && a.y < b.y);
6     });
7     int n = p.size(), k = 0;
8     vector<P> h(2 * n);
9     for (int i = 0; i < n; i++) { // build lower hull
10         while (k >= 2 && (h[k-1] - h[k-2]).cross(p[i] - h[k-2]) <= 0) k--;
11         h[k++] = p[i];
12     }
13     for (int i = n - 2, t = k + 1; i >= 0; i--) { // build upper hull
14         while (k >= t && (h[k-1] - h[k-2]).cross(p[i] - h[k-2]) <= 0) k--;
15         h[k++] = p[i];
16     }
17     h.resize(k - 1); // last point == first, drop it
18     return h;
19 }
20
21 /* ---- Examples ----
22     Want the SMALLEST fence wrapping all points? -> use convexHull
23         square corners (0,0),(4,0),(4,4),(0,4) + inner point (2,2)
24         -> hull = just the 4 corners (inner point dropped)
25     Great for: farthest pair, min enclosing boundary, outer perimeter/area.
26 */
```

## Angles & Rotation

Angle, angle-between, rotate point

```
1  const double PI = acos(-1.0);
2
3  // Angle (radians) of vector v measured from +x axis, in (-PI, PI].
4  double angle(P v) { return atan2(v.y, v.x); }
5
6  // Angle between vectors a and b (0..PI) via the dot product.
7  double angleBetween(P a, P b) {
8      return acos(a.dot(b) / (a.norm() * b.norm()));
9  }
10
11 // Rotate point p around the origin by th radians, counter-clockwise.
12 P rotate(P p, double th) {
13     return { p.x * cos(th) - p.y * sin(th),
14             p.x * sin(th) + p.y * cos(th) };
15 }
16
17 /* ---- Examples ----
18     Want the HEADING/direction of a vector?      -> use angle
19         angle({0,1}) == PI/2
20     Want the angle BETWEEN two directions?      -> use angleBetween
21         angleBetween({1,0},{0,1}) == PI/2    (90 degrees)
22     Want to SPIN a point around the origin?      -> use rotate
23         rotate({1,0}, PI/2) == {0,1}    (quarter turn CCW)
24 */
```

## Shape Formulas Triangle, circle, polygon, sphere

```
1  const double PI = acos(-1.0);
2
3  // ----- Triangle -----
4  double triArea(double base, double h) { return 0.5 * base * h; } // 1/2 b h
5
6  // Heron's formula: area from the three side lengths.
7  double heron(double a, double b, double c) {
8      double s = (a + b + c) / 2.0;           // semi-perimeter
9      return sqrt(s * (s - a) * (s - b) * (s - c));
10 }
11
12 // Area from coordinates = half the cross product magnitude.
13 double triAreaPts(P a, P b, P c) {
14     return fabs((b - a).cross(c - a)) / 2.0;
15 }
16
17 // ----- Circle -----
18 double circleArea(double r) { return PI * r * r; } // pi r^2
19 double circleCirc(double r) { return 2 * PI * r; } // 2 pi r
20
21 // ----- Regular polygon (n sides, side length s) -----
22 double regPolyArea(int n, double s) {
23     return (n * s * s) / (4.0 * tan(PI / n));
24 }
25
26 // ----- Sphere -----
27 double sphereVol(double r) { return 4.0 / 3.0 * PI * r * r * r; } // 4/3 pi r^3
28 double sphereArea(double r) { return 4.0 * PI * r * r; } // 4 pi r^2
29
30 /* ---- Examples ----
31     Triangle area from base & height?    -> triArea(6, 4)    == 12
32     Triangle area from 3 sides only?     -> heron(3, 4, 5)   == 6
33     Triangle area from 3 corners?        -> triAreaPts(a,b,c)
34     Circle area of radius 1?             -> circleArea(1)    == PI
35     Regular hexagon of side 2?          -> regPolyArea(6, 2)
36     Volume of a ball of radius 3?        -> sphereVol(3)
37 */
```

## Ordered Set pbds tree: order\_of\_key / find\_by\_order

---

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 using ordered_set =
tree<int,null_type,less<int>,rb_tree_tag,tree_order_statistics_node_update>;
```

## Ordered Multiset pbds multiset wrapper with helpers

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 class OrderedSet {
5 private:
6     typedef tree<long long,
7                 null_type,
8                 less_equal<long long>,
9                 rb_tree_tag,
10                tree_order_statistics_node_update> ordered_set;
11     ordered_set s;
12 public:
13     void Insert(long long x) {
14         s.insert(x);
15     }
16     bool Exist(long long x) {
17         if (s.upper_bound(x) == s.end()) {
18             return false;
19         }
20         return (*s.upper_bound(x)) == x;
21     }
22     void Erase(long long x) {
23         if (Exist(x)) {
24             s.erase(s.upper_bound(x));
25         }
26     }
27     int Idx(long long x) {
28         if (!Exist(x)) {
29             return -1;
30         }
31         return s.order_of_key(x);
32     }
33     int LastIdx(long long x) {
34         if (!Exist(x)) {
35             return -1;
36         }
37         if (*s.find_by_order(s.size() - 1) == x) {
38             return s.size() - 1;
39         }
40         return Idx(*s.lower_bound(x)) - 1;
41     }
42     long long Value(int index) {
43         return *s.find_by_order(index);
44     }
45     int Count(long long x) {
```

```

46     if (!Exist(x)) {
47         return 0;
48     }
49     return LastIdx(x) - Idx(x) + 1;
50 }
51 void Clear() {
52     s.clear();
53 }
54 int Size() {
55     return s.size();
56 }
57 int Lower_bound(int x) {
58     return *s.upper_bound(x);
59 }
60 int Upper_bound(int x) {
61     return *s.lower_bound(x);
62 }
63 };

```



## Bit Tricks

### Bitwise Operations get / set / clear / toggle + built-in tricks

```

1  // Bit at position i (0-indexed from the least significant bit).
2  int  getBit  (int x, int i) { return (x >> i) & 1; }           // read bit i
3  int  setBit  (int x, int i) { return x | (1 << i); }           // turn bit i ON
4  int  clearBit(int x, int i) { return x & ~(1 << i); }          // turn bit i OFF
5  int  toggle  (int x, int i) { return x ^ (1 << i); }          // flip bit i
6  bool isPow2  (int x)      { return x > 0 && (x & (x - 1)) == 0; }
7
8  // Handy GCC built-ins:
9  //  __builtin_popcount(x) -> number of set bits
10 //  __builtin_ctz(x)      -> trailing zeros (index of lowest set bit)
11 //  __builtin_clz(x)      -> leading zeros (from the top)
12
13 /* ---- Examples ----
14    Want to turn a switch/flag ON?    -> use setBit
15        setBit(0b0100, 0) == 0b0101
16    Want to read one bit?              -> use getBit
17        getBit(0b0101, 0) == 1
18    Want to test a power of two?       -> use isPow2
19        isPow2(16) == true , isPow2(18) == false
20    Want to count set bits?            -> __builtin_popcount(7) == 3
21 */

```

## Bitmask Techniques

Subset & submask enumeration for DP

```
1 // Enumerate ALL subsets of an n-element set: masks 0 .. 2^n - 1.
2 void allSubsets(int n) {
3     for (int mask = 0; mask < (1 << n); mask++) {
4         for (int i = 0; i < n; i++)
5             if (mask >> i & 1) { /* element i is in this subset */ }
6     }
7 }
8
9 // Enumerate all SUBMASKS of a fixed mask (used in subset-sum DP).
10 void allSubmasks(int mask) {
11     for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
12         /* sub is a non-empty submask of mask */
13     }
14 }
15
16 /* ---- Examples ----
17     Want to try EVERY subset of n items?   -> loop mask 0..(1<<n)-1
18         n = 3   -> 8 subsets: 000, 001, 010, ..., 111
19     Want every submask of a set (for DP)? -> use the submask loop
20         mask = 0b101   -> 101, 100, 001
21     Core of: bitmask DP, TSP, assignment problems.
22 */
```

## XOR Tricks XOR properties, unique number, range xor

```
1 // Key XOR properties (memorize these):
2 //   a ^ a = 0           (a value cancels itself)
3 //   a ^ 0 = a
4 //   XOR is commutative & associative -> order does not matter
5
6 // Every element appears an even number of times except ONE - find it.
7 int findUnique(vector<int>& v) {
8     int x = 0;
9     for (int a : v) x ^= a;    // pairs cancel, the odd one survives
10    return x;
11 }
12
13 // XOR of 0 ^ 1 ^ 2 ^ ... ^ n in O(1) using the pattern of n % 4.
14 long long xorUpTo(long long n) {
15     switch (n % 4) {
16         case 0: return n;
17         case 1: return 1;
18         case 2: return n + 1;
19         default: return 0;
20     }
21 }
22
23 /* ---- Examples ----
24     Everyone appears twice except one - find it? -> use findUnique
25         findUnique({4, 1, 2, 1, 2}) == 4
26     Want 1 ^ 2 ^ ... ^ n instantly? -> use xorUpTo
27         xorUpTo(5) == 1           // 1^2^3^4^5
28     Range xor [L..r] = xorUpTo(r) ^ xorUpTo(L - 1).
29 */
```

## Bit Counting & Log2 Highest/lowest bit, popcount, next pow2

```
1 // Bit lengths & positions (for 32-bit int; use the ...ll versions for long long).
2 int highestBit(int x) { return 31 - __builtin_clz(x); } // index of top set bit
3 int lowestBit (int x) { return __builtin_ctz(x); }      // index of bottom set
bit
4 int countBits (int x) { return __builtin_popcount(x); } // number of 1 bits
5 int msbValue  (int x) { return 1 << highestBit(x); }    // value of the highest
bit
6
7 // Round up to the next power of two that is >= x.
8 unsigned nextPow2(unsigned x) {
9     if (x <= 1) return 1;
10    return 1u << (32 - __builtin_clz(x - 1));
11 }
12
13 /* ---- Examples ----
14    Want floor(log2(x)) / index of the top bit? -> use highestBit
15        highestBit(18) == 4      // 18 = 10010b, top bit at index 4
16    Want how many bits are set? -> use countBits
17        countBits(7) == 3
18    Want the next power of two >= x? -> use nextPow2
19        nextPow2(17) == 32
20 */
```

## Bit Hacks Even/odd, swap, branch-free min/abs

```
1 // Small branch-free / no-temp tricks:
2 //   x & 1           -> is x odd? (1 = odd, 0 = even)
3 //   x << k          -> multiply by 2^k
4 //   x >> k          -> divide by 2^k (floor)
5 //   x ^= y; y ^= x; x ^= y;  -> swap two ints without a temp variable
6
7 // Min / max of two ints without an if-branch.
8 int minNoIf(int a, int b) { return b ^ ((a ^ b) & -(a < b)); }
9 int maxNoIf(int a, int b) { return a ^ ((a ^ b) & -(a < b)); }
10
11 // Absolute value without branching (32-bit).
12 int absNoIf(int x) { int m = x >> 31; return (x + m) ^ m; }
13
14 /* ---- Examples ----
15     Want to test even / odd fast?    -> x & 1
16         (6 & 1) == 0 (even) , (7 & 1) == 1 (odd)
17     Want *2 or /2 quickly?          -> shifts
18         (5 << 1) == 10 ; (20 >> 2) == 5
19     Want branch-free min / abs?     -> minNoIf / absNoIf
20         minNoIf(3, 8) == 3 ; absNoIf(-9) == 9
21 */
```

## Binary → Gray Code $gray = i \wedge (i \gg 1)$

```
1  /**
2   * @file gray_code.cpp
3   * @brief Solution for the Gray Code sequence generation problem.
4   * * Binary to Gray Code Formula:
5   * -----
6   * Mathematical Rule:
7   *  $G_n = B_n$  (Most Significant Bit stays the same)
8   *  $G_i = B_{i+1} \text{ XOR } B_i$  (Each remaining bit is XORed with the bit to its
Left)
9   * * Bitwise Implementation:
10  *  $gray = i \wedge (i \gg 1)$ ;
11  * * Constraints:  $1 \leq n \leq 16$ 
12  * Time Complexity:  $O(2^n)$ 
13  * Space Complexity:  $O(1)$ 
14  */
15 #include <iostream>
16 using namespace std;
17 int main() {
18     ios_base::sync_with_stdio(false); cin.tie(NULL);
19     int n;
20     cin >> n;
21     // total_rows =  $2^n$  computed using bitwise left shift
22     int total_rows = 1 << n;
23     // Iterate through all decimal indices from 0 to ( $2^n - 1$ )
24     for (int i = 0; i < total_rows; i++) {
25         // Apply the Bitwise Formula:  $i \text{ XOR } (i \text{ Right Shift } 1)$ 
26         int gray = i ^ (i >> 1);
27         // Extract and print bits from MSB to LSB
28         for (int bit = n - 1; bit >= 0; bit--) {
29             cout << ((gray >> bit) & 1);
30         }
31         cout << "\n";
32     }
33     return 0;
34 }
```